

Material de Referencia

Actividad: Análisis de Scripts Bash en Ciberseguridad

Programación para Ciberseguridad — Fase II — FCFM UANL — AD2026

Este material se entrega impreso. Durante la actividad, solo aparecen las preguntas en pantalla. Analiza cada script con tu equipo antes de responder. No es necesario ejecutar el código: las preguntas se responden leyendo y razonando.

Script 1 — portScanner.sh — Scanner de puertos abiertos

Uso: ./portScanner.sh

```
#!/bin/bash
host=$1 # IP objetivo
firstport=$2 # Puerto inicial
lastport=$3 # Puerto final

function portscan {
for ((counter=$firstport; counter<=$lastport; counter++))
do
(echo >/dev/tcp/$host/$counter) > /dev/null 2>&1 \
&& echo "$counter open"
done
}

portscan # Llamada a la función
```

■ Escanea puertos usando /dev/tcp sin herramientas externas.

Script 2 — netScan.sh — Scanner de IPs activas en la red

Uso: ./netScan.sh

```
#!/bin/bash
function is_alive_ping() {
ping -c 1 $1 > /dev/null 2>&1
[ $? -eq 0 ] && echo "Node with IP: $i is up."
}

for i in 10.0.110.{1..255}
do
is_alive_ping $i & disown
done
```

■ Detecta dispositivos activos en la red mediante ping en paralelo.

Script 3 — osdetect.sh — Detección del sistema operativo

Uso: ./osdetect.sh

```
#!/bin/bash

if type -t wevtutil &> /dev/null # Verifica si es Windows
then
OS=MSWin
elif type -t scutil &> /dev/null # Verifica si es macOS
then
OS=macOS
else
OS=Linux # Por defecto
fi

echo $OS
```

■ Detecta el SO según los comandos disponibles en el sistema.

Script 4 — getlocal.sh — Recoleccion de informacion del sistema

Uso: bash getlocal.sh < cmds.txt

```
#!/bin/bash

function SepCmds() {
    LCMD=${ALINE%%|*} # Extrae comando Linux
    REST=${ALINE#*|}
    WCMD=${REST%%|*} # Extrae comando Windows
    REST=${REST#*|}
    TAG=${REST%%|*} # Extrae etiqueta XML
    if [[ $OSTYPE == "MSWin" ]]; then
        CMD="$WCMD"
    else
        CMD="$LCMD"
    fi
}

function DumpInfo() {
    printf '<systeminfo host="%s" type="%s"' "$HOSTNAME" "$OSTYPE"
    printf ' date="%s" time="%s">\n' "$(date '+%F')" "$(date '+%T')"
    readarray CMDS
    for ALINE in "${CMDS[@]"; do
        if [[ ${ALINE:0:1} == '#' ]]; then continue; fi
        SepCmds
        if [[ ${CMD:0:3} == N/A ]]; then continue; fi
        printf "<%s>\n" $TAG
        $CMD
        printf "</%s>\n" $TAG
    done
    printf "</systeminfo>\n"
}

OSTYPE=$(./osdetect.sh)
HOSTNAME=$(hostname)
TMPFILE="${HOSTNAME}.info"
DumpInfo > $TMPFILE 2>&1
```

■ *Ejecuta comandos definidos en cmds.txt y genera un reporte en formato XML.*

Script 5 — integridad.sh — Verificación de integridad de archivos

Uso: ./integridad.sh

```
#!/bin/bash
ruta=$1
fecha=$(date +%Y-%m-%d %H:%M:%S)
registro="integridad.csv"

for archivo in $(find $ruta -type f); do
    hash=$(sha256sum "$archivo" | awk '{print $1}')
    echo "$archivo,$hash,$fecha" >> $registro
done
```

■ *Calcula SHA256 de todos los archivos en una ruta y guarda los resultados con fecha.*

Script 6 — alertaPuertos.sh — Alerta de puertos no autorizados

Uso: ./alertaPuertos.sh

```
#!/bin/bash
ip=$1
fecha=$(date +%Y-%m-%d_%H-%M)
reporte="nmap_$fecha.txt"
alertas="alertas_$fecha.txt"
puertos_autorizados=("22" "80" "443")

nmap -p- --open $ip > "$reporte"

puertos_detectados=$(grep -Eo '[0-9]+/tcp' "$reporte" | cut -d '/' -f1)

for puerto in $puertos_detectados; do
    autorizado=false
    for permitido in "${puertos_autorizados[@]}"; do
        if [ "$puerto" -eq "$permitido" ]; then
            autorizado=true
            break
        fi
    done
    if [ "$autorizado" = false ]; then
        echo "Puerto no autorizado: $puerto" | tee -a "$alertas"
    fi
done
```

■ *Escanea todos los puertos y alerta si detecta puertos abiertos no autorizados.*

Script 7 — Fragmento A — Pipeline de auditoria de permisos (R11)

Fragmento del compendio de Fase II — Reto R11

```
# Pipeline de auditoria de permisos
ls -l | grep -E "^.{8}w" | awk '{print $NF}' | \
sed 's/^[RIESG0] Escritura abierta: /' > reporte.txt

# Ver el reporte generado
cat reporte.txt

# Adicionalmente: buscar archivos con bit SUID activo
find / -type f -perm -4000 2>/dev/null | head -10
```

■ *Detecta archivos con permiso de escritura abierto para "otros" y genera un reporte.*

Script 8 — Fragmento B — Analisis de auth.log (R21)

Fragmento del compendio de Fase II — Reto R21

```
# Extraer y contar intentos de login fallidos por IP/usuario
grep "Failed password" /var/log/auth.log | \
awk '{printf "{\"ip\":\"%s\",\"usuario\":\"%s\"}\n", $11, $9}' | \
sort | uniq -c | sort -rn | head -5

# Generar alerta si una IP supera el umbral de intentos
UMBRAL=5
grep "Failed password" /var/log/auth.log | \
awk '{print $11}' | sort | uniq -c | \
while read -r cuenta ip; do
if [ "$cuenta" -ge "$UMBRAL" ]; then
echo "ALERTA: $ip tiene $cuenta intentos fallidos"
fi
done
```

■ *Extrae IPs y usuarios con intentos de login fallidos y los ordena por frecuencia.*